Battery Monitor for Goosebot

EEL 4312 Electric and Hybrid Vehicles – Dr. Hoan

4/20/2026

Christian Cruz, Luis Mata, and Zander Butler

Florida Polytechnic University

Table of Contents

Abstract 2
Introduction 3
Materials & Methods 4
Results 7
Discussion 9
Conclusion 10
Acknowledgement 11
References 11
GitHub Repo 11

Abstract

This report presents the design and implementation of a battery monitoring system developed for the Goosebot, an electrically powered robotic platform. The system was designed to measure the Goosebot's 12V battery voltage in real time and display the remaining charge as a percentage to the user. To accomplish this, a voltage divider circuit composed of resistors was used to scale the battery voltage down to a safe readable range for an Arduino Uno microcontroller, which was programmed using the Arduino IDE to process the voltage readings and calculate the corresponding battery percentage. The calculated values were then displayed on a 20x4 LCD screen connected to the Arduino via the I2C communication interface, providing the user with a clear and continuously updated readout of both the battery voltage and remaining charge percentage. Testing was conducted by comparing the Arduino's voltage readings against measurements taken with a multimeter to verify the accuracy of the system. The results demonstrated that the system was capable of reliably monitoring and displaying the Goosebot's battery level, offering a practical and low-cost solution for real-time battery management in small robotic applications.

Introduction

The Goosebot Battery Monitor System is a project designed to provide real-time battery level feedback for the Goosebot platform. As battery management is a critical aspect of any electrically powered system, the ability to accurately monitor and display remaining charge ensures safe and efficient operation. This project implements a hardware-software solution using an Arduino Uno microcontroller, a voltage divider circuit, and a 20x4 LCD display to measure the 12V battery's voltage, convert it to a human-readable percentage between 0 and 100%, and present that information to the user in real time. The Arduino Uno was programmed using the Arduino IDE, which allowed the team to develop and upload the code responsible for reading the stepped-down voltage signal, performing the percentage conversion calculation, and driving the LCD display output. By stepping down the battery voltage through a resistor-based voltage divider to a range safely readable by the Arduino's analog input pins, the system is able to accurately sample the battery's current charge level. The converted percentage value is then formatted and sent to the 20x4 LCD display, giving the user a clear and continuously updated readout of the battery's remaining capacity across its larger screen area. The following report details the design requirements, materials, methods, testing procedures, and team contributions involved in bringing this system to completion.

Materials & Methods

Bill of Materials:

Materials	Prices
Arduino Uno Board	**$21**
20 x 4 LCD Display	**$8**
Kit of Resistors	**$5**
HOOVO 3S 11.1V Lipo Battery 2200mAh 50C	**$36**
Breadboard (840 Pins)	**$10**
Wire Kit (Male to Male and Female to Male)	**$7**
Total	**$87**

Procedure:

To begin the implementation of the Goosebot Battery Monitor System, the team first developed and uploaded the necessary code to the Arduino Uno using the Arduino IDE software. The program was written using two libraries: the Wire.h library, which enables I2C communication between devices, and the LiquidCrystal_I2C library, which handles the control of the LCD display. The LCD was initialized at I2C address 0x27 and pin A0 was designated as the input pin for reading the battery voltage signal. In the setup function, the Serial Monitor was initialized at a baud rate of 9600 for debugging purposes, and the LCD was powered on with its backlight enabled. The main loop was programmed to read the raw voltage value coming into pin A0, convert it into an actual battery voltage using the Arduino's 5V reference and a voltage divider ratio of 7.06, and then calculate a battery percentage by comparing the measured voltage against a minimum threshold of 10.5V and a maximum threshold of 12.6V. The resulting percentage was constrained between 0 and 100% to prevent any out-of-range readings from being displayed. The voltage and percentage values were also continuously sent to the Serial Monitor once per second, which proved useful for verifying the accuracy of readings during testing.

The voltage divider circuit was constructed to step down the Goosebot's 12V battery voltage to a level safe for the Arduino Uno's analog input pins, which can only read voltages up to 5V. The circuit was built using two 10kΩ resistors connected in series, with a 3.3kΩ resistor connected in parallel with the lower of the two resistors. This combination produces a voltage divider ratio of approximately 7.06, which scales the battery voltage down proportionally while preserving the relationship needed to accurately calculate the true battery voltage in the code. The positive wire from the battery was connected to the top of the voltage divider circuit, and the negative wire was connected to the GND pin on the Arduino Uno, establishing a common ground for the whole system.

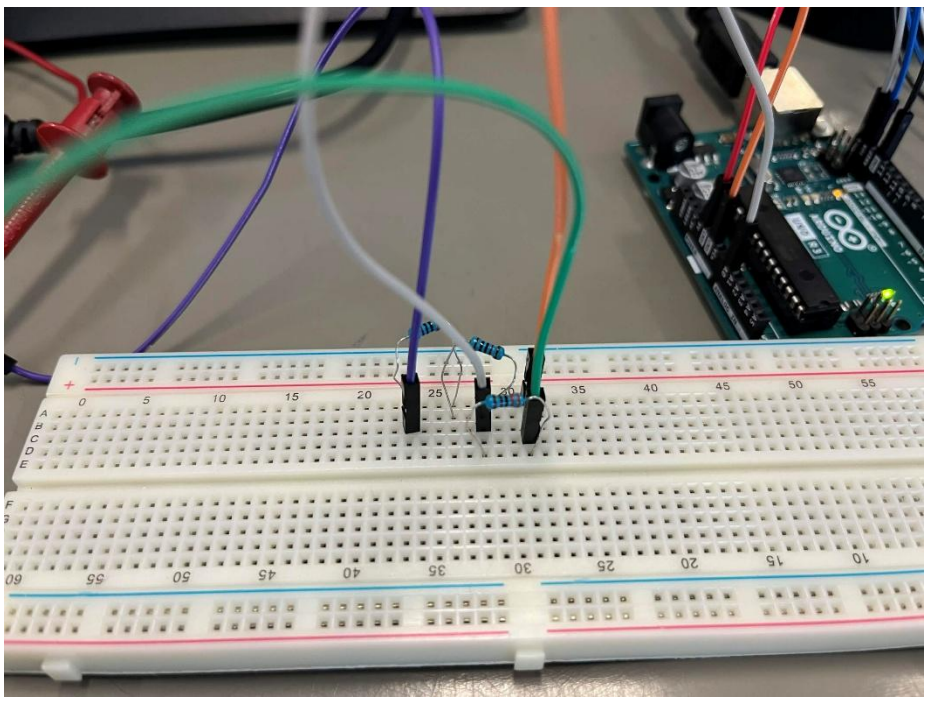

Figure 1. Overview of the Voltage Divider on the Breadboard

The scaled-down voltage was then fed from the junction of the two series resistors into the A0 analog input pin on the Arduino Uno. The Arduino sampled this voltage once every second, as set by the 1000 millisecond delay in the code, and applied the conversion formula to calculate the corresponding battery percentage. Finally, the 20x4 LCD display was connected to the Arduino Uno through the I2C interface using the required power, ground, and data pins. The display received the processed voltage and percentage values from the Arduino and presented them across

two clearly labeled rows, giving the user a straightforward and continuously updated readout of the Goosebot's remaining battery level.

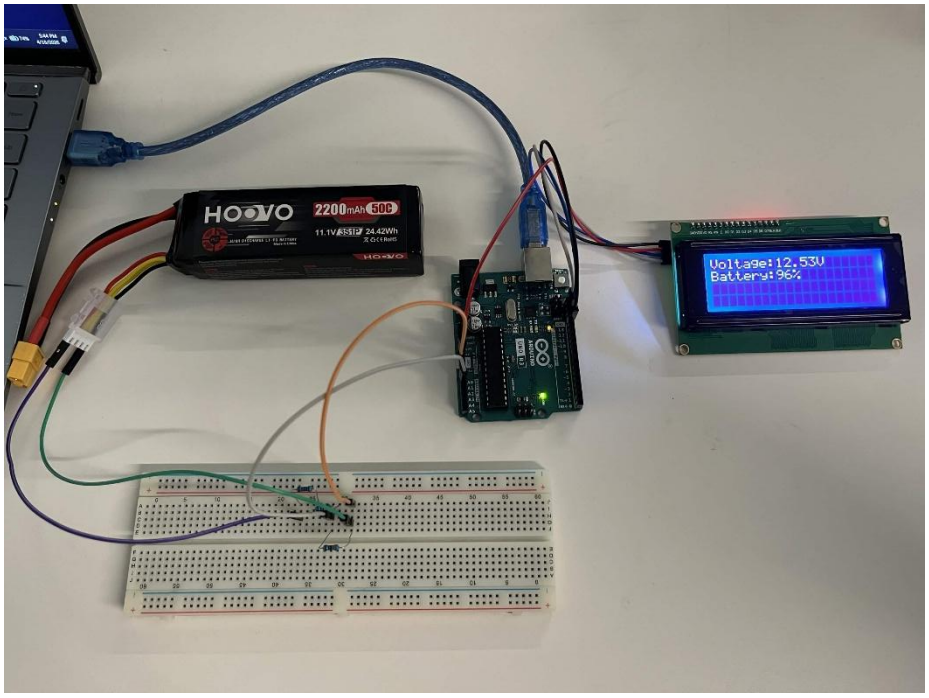

Figure 2. Complete Battery Monitor with Connections to Battery, Arduino Uno and LCD Display

Results

The implemented battery monitoring system produced successful results across all major design requirements. The voltage divider circuit performed as intended, effectively scaling the Goosebot's 12V battery voltage down to a safe range readable by the Arduino Uno's analog input pin, which was then processed and output to the 20x4 LCD display. The Serial Monitor, as seen in Figure 3, was instrumental during testing by providing real-time feedback of the raw ADC values and the divided voltage readings, allowing the team to verify that the Arduino was accurately interpreting the incoming voltage signal and calculating the correct battery percentage.

```
Output    Serial Monitor ×
Message (Enter to send message to 'Arduino UNO' on 'COM3')

RAW ADC: 361 | Divider Voltage: 1.76 V | Battery Voltage: 12.46 V | Battery %: 93%
RAW ADC: 361 | Divider Voltage: 1.76 V | Battery Voltage: 12.46 V | Battery %: 93%
RAW ADC: 362 | Divider Voltage: 1.77 V | Battery Voltage: 12.49 V | Battery %: 94%
RAW ADC: 361 | Divider Voltage: 1.76 V | Battery Voltage: 12.46 V | Battery %: 93%
RAW ADC: 362 | Divider Voltage: 1.77 V | Battery Voltage: 12.49 V | Battery %: 94%
RAW ADC: 361 | Divider Voltage: 1.76 V | Battery Voltage: 12.46 V | Battery %: 93%
RAW ADC: 361 | Divider Voltage: 1.76 V | Battery Voltage: 12.46 V | Battery %: 93%
RAW ADC: 363 | Divider Voltage: 1.77 V | Battery Voltage: 12.53 V | Battery %: 96%
RAW ADC: 362 | Divider Voltage: 1.77 V | Battery Voltage: 12.49 V | Battery %: 94%
RAW ADC: 361 | Divider Voltage: 1.76 V | Battery Voltage: 12.46 V | Battery %: 93%
```

Figure 3. Serial Monitor Output

The system was configured to map the battery percentage across a voltage range of 10.5V at 0% and 12.6V at 100%, with these thresholds chosen to reflect the safe operating range of the Goosebot's battery and prevent over-discharge that could cause damage to the cells. As seen in Figures 4 and 5, the LCD display correctly presented the battery voltage and its corresponding percentage, updating in real time as the voltage fluctuated. The readings from the Arduino were also compared against measurements taken with a multimeter, confirming that the system was producing accurate and consistent results. Overall, the system successfully met its goal of providing a reliable and user-friendly battery level display for the Goosebot platform.

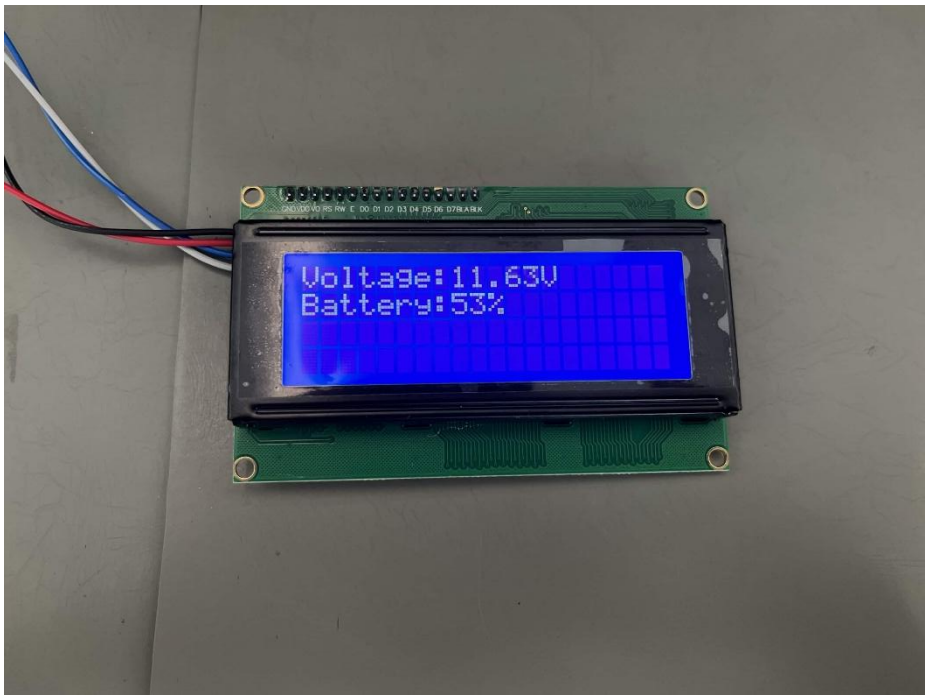

Figure 4. LCD Displaying Battery Percentage at 53%

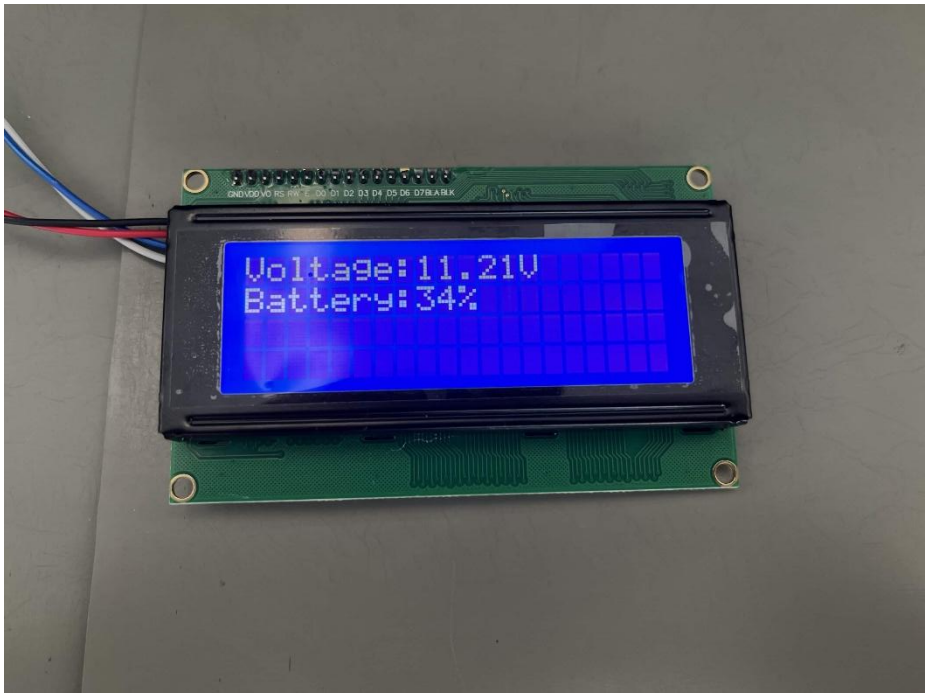

Figure 5. LCD Displaying Battery Percentage at 34%

Discussion

The results of this project demonstrate that a functional and accurate battery monitoring system can be successfully implemented for the Goosebot platform using readily available and low-cost components. The voltage divider circuit proved to be an effective solution for stepping down the 12V battery voltage to a range compatible with the Arduino Uno's analog input, and the code developed in the Arduino IDE reliably translated those readings into meaningful battery percentage values displayed on the 20x4 LCD screen.

One of the more critical aspects of the design was selecting the appropriate voltage thresholds for the battery percentage calculation. Setting the minimum voltage at 10.5V and the maximum at 12.6V ensured that the displayed percentage accurately reflected the usable capacity of the battery while also protecting it from being discharged to a damaging level, which is consistent with the safe operating range of a standard 12V battery.

The use of the Serial Monitor during testing proved to be a valuable tool, as it allowed the team to observe the raw ADC values, divided voltage, and calculated percentages in real time. This made it significantly easier to identify and correct any discrepancies between the expected and actual voltage readings, and the subsequent comparison against multimeter measurements further validated the accuracy of the system.

While the system performed well overall, there are areas that could be improved in future iterations. The passive voltage divider draws a small continuous current from the battery, which could contribute to minor inaccuracies over extended periods. Additionally, incorporating a dedicated battery management IC could provide greater precision and added protection features, while expanding the LCD output to include a low battery warning or estimated remaining runtime would further enhance the system's usability for the operator.

Conclusion

In conclusion, the Goosebot Battery Monitor System successfully met its core design objectives of measuring, converting, and displaying the Goosebot's battery level in real time. The project brought together fundamental concepts in circuit design, embedded programming, and battery management to produce a functional and practical solution built from low-cost, accessible components. Each member of the team contributed meaningfully to the project through their assigned roles in hardware setup, programming, and testing, and the collaborative effort resulted in a system that performed reliably throughout the validation process.

From a broader perspective, this project highlighted just how important battery monitoring is in the context of electric and hybrid vehicle systems. Without a reliable means of tracking battery state, an electrically powered platform risks unexpected shutdowns or long-term battery damage from over-discharge, both of which can have significant consequences in real-world applications. The Goosebot Battery Monitor System addresses this concern in a direct and effective way, and the principles applied here scale well to larger and more complex electric vehicle systems.

Overall, this project served as a valuable hands-on learning experience that connected classroom concepts to a real engineering problem. The team gained practical knowledge in embedded systems design, analog signal processing, and the application of battery management principles, all of which are directly relevant to the field of electric and hybrid vehicles.

Acknowledgement

The team would like to express their gratitude to Dr. Hoan Ngo for his guidance and instruction throughout the course of the project. His teaching provided the foundational knowledge that made this project possible and helped shape the team's understanding of battery management and electric vehicle systems. The team would also like to acknowledge the support and resources made available throughout the course, which were instrumental in the successful design, construction, and testing of the Goosebot Battery Monitor System.

References

[1] G. L. Plett, Battery Management Systems, Volume II: Equivalent-Circuit Methods. Norwood, MA: Artech House, 2016.

[2] J. Larminie, Electric Vehicle Technology Explained, 2nd ed. New Delhi, India: Wiley India Pvt. Ltd., 2015.

[3] I. Husain, Electric and Hybrid Vehicles: Design Fundamentals, 3rd ed. Boca Raton, FL: CRC Press, 2021.

[4] funshort1234, "How to measure voltage using Arduino," YouTube Shorts, Feb. 12, 2025. [Online]. Available: https://youtube.com/shorts/KcBC3Jm7Rjw?si=qrgzWjWKxemQXLtO

GitHub Repo

Link to Repository: https://github.com/Christian10951/Battery-Monitor-for-Goosebot.git